

Performance Analysis

Sequential vs. Concurrent File I/O

Evaluating Multi-thread Efficiency and Speedup

Experiment Overview

The objective of this experiment is to analyze the performance scalability of an Operating System under heavy I/O pressure by comparing sequential and concurrent execution models. The experiment is designed around a high-frequency repetitive workload involving **10,000 independent files**. Each file undergoes a complete lifecycle: **creation** and **synchronous writing** of 1,000 lines to stress the disk controller, utilizing `rewind()` to reset the file pointer, **reading** the data back into memory using a small 256-byte buffer to maximize system call frequency, and finally **deleting** the file to trigger file system metadata updates. By comparing a single-threaded linear execution against a 4-threaded model using the **POSIX Threads (pthread)** library, the workload highlights the trade-offs between **I/O latency masking** and the constraints imposed by hardware resource contention and kernel-level context switching.

Source Code (Implementation)

```
1 void perform_heavy_io(int id) {
2     char filename[64]; sprintf(filename, "io_test_%d.tmp", id);
3     FILE *f = fopen(filename, "w+");
4     if (!f) return;
5     for (int i = 0; i < 1000; i++) {
6         fprintf(f, "I/O System Call Generation - File ID: %d, Line: %d\n", id, i);
7     }
8     rewind(f);
9     char buffer[256];
10    while (fgets(buffer, sizeof(buffer), f) != NULL);
11    fclose(f);
12    remove(filename);
13 }
14
15 void* thread_routine(void* arg) {
16     int thread_id = *(int*)arg;
17     int files_per_thread = 10000 / 4;
18     int start_idx = thread_id * files_per_thread;
19     for (int i = start_idx; i < start_idx + files_per_thread; i++) {
20         perform_heavy_io(i);
21     }
22     return NULL;
23 }
```

Console Output

```
-----
[INFO] Workload: 10000 files, 1000 lines
-----
[PROCESS] Running Sequential Execution..
[INFO] Sequential Duration: 22.9350 s

[PROCESS] Running 4T Parallel Execution..
[INFO] Parallel Duration: 10.8978 s
```

Performance Comparison

Execution	Threads	Time (s)
Sequential	1	22.9350
Concurrent	4	10.8978
Speedup		2.10x

*Speedup calculated as T_{seq}/T_{par}
Measured using monotonic high-resolution timers.

Analysis & Discussion

Findings: The multi-threaded version achieved a speedup of **2.10x**. While using 4 threads significantly reduced the execution time, the result is lower than the theoretical 4x gain.

- I/O Latency Hiding:** Concurrency allows the OS to overlap the waiting periods of different threads. While one thread is blocked by a synchronous I/O call, another thread can utilize the CPU to initiate its own request, increasing overall throughput.
- Disk Bottleneck:** The primary constraint is physical hardware. Since all threads compete for the same disk controller and storage bandwidth, resource contention prevents perfect linear scaling.
- System Call Overhead:** The high frequency of file operations increases kernel-mode transitions. The overhead of managing the pthread lifecycle and context switches acts as a "management tax" on performance.

Final Conclusion: Concurrency is highly effective for masking I/O latency, but the ultimate performance ceiling is determined by the physical throughput of the storage hardware rather than the number of CPU cores.